UNIVERSITAS ACHMAD YANI (UAY) • INFRASTRUKTUR LAYANAN BERKAS TERPUSAT

USULAN FITUR (FEATURE REQUEST / RFC)

Manajemen Direktori Hierarkis Dinamis untuk Klien Aplikasi pada UAY File Service

Nomor Dokumen	**RFC-UAY-FS-2026-001**	**Tanggal Pengajuan**	13 Oktober 2026
Pengusul / Pemohon	Tim Pengembang E-Learning UAY	**Penerima / Sasaran**	Tim Pengembang & Pengelola UAY File Service
Status Dokumen	**PROPOSED (Menunggu Tinjauan Teknis)**	**Tingkat Prioritas**	**TINGGI (High / Usability & Scalability)**

1. Ringkasan Eksekutif & Latar Belakang Masalah

Dalam operasional integrasi akademik antara platform **E-Learning UAY** dan **UAY File Service**, seluruh berkas digital—mulai dari dokumen materi perkuliahan, modul presentasi, rekaman video, pengumpulan tugas mahasiswa, hingga berkas jawaban kuis—diunggah melalui antarmuka REST API File Service.

Saat ini, seluruh berkas yang dikirimkan oleh klien aplikasi tersimpan di tingkat **root repository** (dengan nilai `folder_id = null`). Mengingat proyeksi volume unggahan mencapai puluhan ribu berkas per semester perkuliahan, penumpukan berkas di level *root* menimbulkan beberapa hambatan kritis:

- **Kesulitan Audit dan Navigasi Visual:** Pada antarmuka *File Explorer Admin Console* File Service, ribuan berkas dari berbagai program studi, semester, dan jenis aktivitas bercampur dalam satu daftar tanpa pengelompokan folder.
- **Ketiadaan Isolasi Logis Akademik:** Struktur repositori File Service tidak mencerminkan hirarki tahun akademik maupun mata kuliah, sehingga pelacakan manual membutuhkan penelusuran balik ke database E-Learning.
- **Keterbatasan Antarmuka M2M Saat Ini:** Endpoint upload aplikasi klien (`POST /v1/repositories/:repositoryId/files`) menerima parameter `folderId` , namun File Service **belum menyediakan endpoint bagi aplikasi klien (Machine-to-Machine / API Key) untuk membuat folder atau membuat jalur direktori secara dinamis**. Endpoint pembuatan folder saat ini hanya ada di rute Console (`POST /v1/console/repositories/:repositoryId/folders`) yang memerlukan sesi login pengguna SSO interaktif di peramban.

2. Analisis Arsitektur File Service Saat Ini

Berdasarkan audit teknis terhadap basis kode dan skema basis data UAY File Service:

1. **Model Basis Data Sudah Mendukung Struktur Pohon (Recursive Folder Tree):**
   Tabel `folders` pada PostgreSQL telah memiliki relasi rekursif (`parent_folder_id → folder_id`) dengan indeks unik `(repository_id, name)` di root dan `(repository_id, parent_folder_id, name)` untuk sub-folder. Tabel `files` juga telah memiliki relasi `folder_id` yang terhubung langsung ke tabel `folders` .
2. **Keterbatasan Batas Parameter Multipart (Multer Limits):**
   Pada service storage lokal (`filesystem-storage.service.ts`), konfigurasi Multer saat ini dibatasi maksimal `fields: 3` (hanya menerima `folderId` , `contextType` , dan `contextId`). Jika klien aplikasi mengirimkan parameter tambahan seperti jalur path, parser Multer akan menghasilkan error `400 Bad Request (LIMIT_FIELD_COUNT)` .
3. **Keterbatasan Hak Akses Endpoint Console:**
   Endpoint pembuatan folder hanya tersedia pada Controller Sandbox Console yang dijaga oleh

`ConsoleBearerAuthGuard` dan `ConsoleRepositoryManagerGuard` (memerlukan audience `KEYCLOAK_CONSOLE_CLIENT_ID`). Klien backend M2M seperti E-Learning (dengan client ID `elearning-uay`) tidak dapat mengakses rute tersebut.

3. Spesifikasi Usulan Solusi

Kami mengusulkan dua alternatif solusi yang dapat dipertimbangkan oleh tim pengembang File Service:

Opsi A (Sangat Direkomendasikan): Dukungan Parameter `folderPath` pada Endpoint Upload

Pendekatan ini paling optimal dan efisien karena bersifat **atomik (1 round-trip HTTP)**, di mana klien aplikasi tidak perlu melakukan pemanggilan terpisah untuk membuat folder sebelum mengunggah berkas.

- **Endpoint:** `POST /v1/repositories/:repositoryId/files` (multipart/form-data).
- **Parameter Baru:** `folderPath` (string opsional), contoh: `"2024-2025 Ganjil/[IF101] Pemrograman Web - TI-3A/01_Materi"`.
- **Mekanisme Kerja Server:**
  1. Jika `folderId` (UUID) disertakan oleh klien, File Service langsung menggunakan folder tersebut.
  2. Jika `folderId` tidak disertakan tetapi `folderPath` ada, File Service secara otomatis memecah segmen path (dipisahkan oleh `/`) dari kiri ke kanan.
  3. File Service mencari folder aktif yang cocok pada setiap tingkat hirarki; jika belum ditemukan, folder dibuat secara otomatis (dengan penanganan konkurensi/idempoten).
  4. Folder tingkat terdalam (*leaf folder*) dijadikan sebagai `folderId` dari berkas yang diunggah.
- **Penyesuaian Multer:** Menaikkan batas `fields: 10` dan `parts: 12` pada `getFileUploadOptions()`.

Opsi B: Penambahan Endpoint Folder untuk Klien Aplikasi (M2M)

Jika arsitektur File Service mengharuskan pemisahan tanggung jawab antara pengelolaan folder dan pengunggahan berkas:

- `GET /v1/repositories/:repositoryId/folders`: Mengambil daftar folder aktif di repositori (izin: `FILE_LIST`).
- `POST /v1/repositories/:repositoryId/folders`: Membuat folder atau memastikan jalur hierarki tersedia (*ensure path*) dengan izin `FILE_UPLOAD`. Endpoint bersifat idempoten (mengembalikan data folder jika sudah ada sebelumnya).

4. Standar Hirarki Direktori E-Learning UAY

Platform E-Learning UAY telah menyiapkan algoritma pemetaan otomatis 3 tingkat yang ramah arsip akademik dan konsisten:

```
[Repository Root]
├── 2024-2025 Ganjil/
│   ├── [IF101] Pemrograman Web - TI-3A/
│   │   ├── 00_Sampul/           (Foto banner & cover kelas)
│   │   ├── 01_Materi/           (Modul PDF, slide presentasi, video pembelajaran)
│   │   ├── 02_Tugas/            (Pengumpulan berkas tugas & laporan praktikum)
│   │   └── 03_Kuis/             (Lembar kerja kuis & jawaban ujian)
│   └── [IF102] Struktur Data - TI-1B/
│       ├── 01_Materi/
│       └── 02_Tugas/
└── Umum/
    └── 04_Lainnya/
```

5. Referensi Implementasi Teknis (NestJS & Prisma)

Berikut adalah cuplikan kode implementasi fungsi resolusi folder yang aman, efisien, dan tahan konkurensi untuk diterapkan di `files.service.ts` File Service:

```
async ensureFolderPath(
  repositoryId: string,
  folderPath: string,
  actorId?: string,
): Promise<string> {
  const segments = folderPath.replace(/\\/g, '/').split('/').map(s => s.trim()).filter(Boolean);
  if (segments.length === 0) throw new BadRequestException('Folder path cannot be empty');
  if (segments.length > 20) throw new BadRequestException('Depth cannot exceed 20 levels');

  // Validasi sanitasi segmen (mencegah directory traversal & karakter ilegal)
  for (const seg of segments) {
    if (seg === '.' || seg === '..' || seg.length > 255 || /[\x00-\x1f]/.test(seg)) {
      throw new BadRequestException(`Invalid folder segment: "${seg}"`);
    }
  }

  let currentParentId: string | null = null;
  for (const seg of segments) {
    let folder = await this.prisma.folder.findFirst({
      where: { repositoryId, parentFolderId: currentParentId, name: seg, status: 'active' },
      select: { folderId: true },
    });

    if (!folder) {
      try {
        folder = await this.prisma.folder.create({
          data: { repositoryId, parentFolderId: currentParentId, name: seg, status: 'active' },
          select: { folderId: true },
        });
      } catch (err: any) {
        // Tangani race condition bila dua upload bersamaan membuat folder yang sama
        if (err?.code === 'P2002') {
          folder = await this.prisma.folder.findFirst({
            where: { repositoryId, parentFolderId: currentParentId, name: seg, status: 'active' },
            select: { folderId: true },
          });
          if (!folder) throw err;
        } else { throw err; }
      }
    }
    currentParentId = folder.folderId;
  }
  return currentParentId!;
}
```

6. Jaminan Kompatibilitas Mundur (Backward Compatibility)

Jaminan Keamanan Sistem:

1. **Klien Lama Tetap Berfungsi Normal:** Jika aplikasi klien tidak mengirimkan `folderPath` atau mengirimkan `folderId: null`, berkas tetap disimpan di root seperti perilaku sebelumnya.
2. **Tanpa Migrasi Basis Data Baru:** Tabel `folders` dan relasinya sudah lengkap sejak migrasi rilis awal. Tidak ada tabel atau kolom baru yang perlu ditambahkan.
3. **Tanpa Penambahan Hak Akses:** Operasi pembuatan direktori memanfaatkan hak akses `FILE_UPLOAD` yang telah terdaftar pada sistem perizinan repositori.
4. **Tahan Terhadap Beban Konkuren:** Penanganan galat Prisma `P2002` memastikan proses upload simultan oleh banyak mahasiswa tidak menghasilkan folder duplikat atau crash.

7. Kesimpulan & Koordinasi Antar-Tim

Pihak Terkait	Tindakan yang Diharapkan
Tim File Service	Meninjau usulan teknis ini, menyetujui implementasi parameter `folderPath` (Opsi A) atau endpoint folder (Opsi B), dan menerapkannya pada siklus rilis File Service berikutnya.
Tim E-Learning	Mengaktifkan pengiriman jalur hierarki `folderPath` otomatis dari sisi E-Learning segera setelah pembaruan File Service diaktifkan di lingkungan staging dan production.